

Week 10 Explained Student Worksheet

Integrated Mini Search Engine

Name _____ Student ID _____ Group _____

Integrate the previous labs into a small command-line search engine. Load real text files, rank a user query, display results, and evaluate queries that have relevance judgments.

Use this worksheet during the two-hour practical session. Read, predict, run, and explain. The existing lab sheet contains the complete program; this companion explains key ideas and provides space to record your own evidence. Python 3.10 or later is sufficient; no pip packages are needed.

What you should be able to do

- Organize the retrieval pipeline into small Python modules.
- Load a local document collection and build an inverted index.
- Calculate TF-IDF and rank user queries with cosine similarity.
- Display document IDs, scores, and text snippets.
- Evaluate judged queries and document limitations.

Understand the concepts

The pipeline is: text files → document loading → cleaning → tokenization → stop-word removal → vocabulary → inverted index → TF-IDF → query processing → cosine similarity → ranking → evaluation. Each stage has an explicit input and output that can be inspected.

file_utils.py loads only the docs folder and reports missing or empty collections. preprocessing.py replaces non-ASCII-letter runs with spaces, then lowercases, tokenizes, and removes the same stop words. For this English teaching collection, it produces the same retained tokens as Week 3.

indexing.py builds unique sorted postings. tfidf.py constructs the vocabulary, TF/DF/IDF, and weighted vectors. ranking.py reuses collection IDF for queries and computes cosine scores. The index is built and can be inspected; the simple dense ranker compares the query with all document vectors. Candidate pruning is an optional extension.

main.py loads and indexes once per session. Interactive searches reuse that model. Only positive-score results are displayed, with at most five documents. Evaluation at k=3 runs when a normalized exact query string has judgments. Unjudged queries still rank, but the program makes no relevance-quality claim for them.

The application uses ASCII word tokens, a fixed stop-word set, raw TF, unsmoothed natural-log IDF, and deterministic ID tie order. It does not implement stemming, phrase search, semantic matching, or a web interface. Rebuild the model after changing the collection, and review the judgments if document contents change.

Predict before running

Name the project stages from reading documents to evaluation, and distinguish the model built once from the work repeated for each query.

Prepare your terminal

Extract the course and open a terminal inside Week_10_IR_Project. Keep docs and data beside the Python files. The main demonstration is main.py; your exercise starter is lab10_exercises.py. Starters run but deliberately contain unfinished TODO functions.

```
py -3.13 main.py
py -3.13 lab10_exercises.py
```

Use py -3 or python instead if that is your working Python 3.10+ command.

Worked example with explanation

Run this complete integration example from inside the Week 10 folder. It uses the included modules and computes its result from docs, rather than hard-coding a ranking.

Run this illustration from the Week_10_IR_Project folder so imports and docs resolve correctly.

```
import os
from file_utils import load_documents
from tfidf import build_tfidf
from ranking import rank_query
documents = load_documents(os.path.join(os.getcwd(), "docs"))
vocabulary, counts, df, idf, vectors = build_tfidf(documents)
results = rank_query("python data analysis", vocabulary, idf, vectors)
for name, score in results[:3]:
    print(name, f"{score:.4f}")
```

Expected output from an executed example

```
doc4 0.5123
doc9 0.4326
doc1 0.3466
```

How each stage produces the result

1. Run this worked example from the Week 10 folder because its short illustration uses os.getcwd() to find docs and imports local modules.
2. load_documents reads the collection. build_tfidf returns the shared vocabulary, frequencies, IDF values, and document vectors.
3. rank_query processes the query with the same tokenizer and weights, then returns ID-score pairs in deterministic order. The displayed top three agree with Week 8 on identical data.
4. The complete main.py uses script-relative paths, builds the model once for repeated queries, displays snippets, and evaluates an exact judged query when matching judgments exist.

Explain one intermediate value in your own words

Read the weekly program

The following excerpt is copied exactly from main.py. It depends on the imports, constants, and helper functions in that file. Read it there rather than pasting it into an empty script and expecting the excerpt to run alone.

```
def show_results(query, model, documents, judgments):
    vocabulary, counts, df, idf, vectors = model
    results = rank_query(query, vocabulary, idf, vectors)
    ranked = [name for name, score in results if score > 0]
    print("\nQuery:", query)
    print("Query tokens:", tokenize(query))
    if not ranked:
        print("No matching documents with a positive score.")
    for position, (name, score) in enumerate(results[:5], 1):
        if score > 0:
            print(f"{position}. {name} {score:.4f}")
            print(" ", documents[name].strip()[:90])
    # Judgments refer to exact query strings, ignoring case/whitespace.
    key = " ".join(query.lower().split())
    for case in judgments:
        if key == " ".join(case["query"].lower().split()):
            report("Evaluation", ranked, case["relevant"], k=3)
            return
    print("No predefined relevance judgments for this query.")
```

Code walkthrough

1. show_results unpacks the model and obtains the ranked pairs. It filters positive-score IDs for evaluation while displaying only positive entries among the top five.
2. The preview takes the first 90 characters of the stored document after stripping outer whitespace. It is a simple snippet, not a query-aware highlighting algorithm.
3. A query key normalizes case and whitespace for comparison with judgment entries. Semantically similar wording is not automatically the same judged query.
4. main builds both postings and TF-IDF. Baseline ranked search uses the vectors; the postings support inspection and an optional separate Boolean extension. Missing or empty collections are reported rather than silently replaced.

If a document is added after model construction, which stored structures must be rebuilt before the next search?

Connection to the course

Your final project should explain not only which files run, but why the preprocessing, coordinate order, scoring, and evaluation choices produce the observed results.

Guided practical activity

Work through the steps in order. Compare your predictions with observations and record at least one specific value per experiment. Keep original baseline data until you have saved the first transcript. Put experiments that change files in a disposable copy.

Step 1 Inspect the ten docs files and three judged queries. Sketch the pipeline and assign each stage to a module.

Assign loading, preprocessing, postings, weighting, ranking, and evaluation to their actual files before editing.

Step 2 Trace `file_utils` and preprocessing; compare their retained tokens with the Week 3 function on the sample collection.

Compare tokens on the same input. A module boundary should not change preprocessing semantics.

Step 3 Build the inverted index and model once. Verify that posting lengths equal the DF counts.

Build reusable model data once, then inspect DF and posting lengths for selected terms.

Step 4 Run `python main.py` for the default demonstration, then `python main.py --interactive` for repeated searches.

Keep the process open for repeated interactive searches. Type `quit` to finish and preserve the transcript.

Step 5 Search the three judged queries and one new query. Explain why evaluation appears only for judged queries.

Search both judged and new queries. An unjudged query should produce the explanatory message rather than invented evaluation metrics.

Step 6 Complete the starter integration using your earlier functions. Test a missing folder, an empty query, an unknown-term query, and a modified collection.

Integrate your earlier functions in the starter. Test missing-data behavior in a disposable copy and rebuild after changing the collection.

Test record

Check to demonstrate	Actual observation	Pass or revise
The default query agrees with Week 8 on unchanged data		
An unknown query reports no positive matches		
A new unjudged query is not given invented metrics		

Record one prediction that changed after running the program and explain why

Independent work and submission

Complete these selected tasks in your own exercise file. Use the existing lab sheet for additional exercises. Do not submit the unchanged starter or simply import a solution in place of implementing the requested function.

- Basic: change the display limit from five results to three without changing the metric cutoff.
- Implementation: finish `search(query, documents)` in the starter using your earlier preprocessing, weighting, and cosine functions.
- Debugging: deliberately change only query preprocessing. Use a test query to show why document/query consistency matters.
- Interpretation: inspect the posting list and DF for data and explain how they support different stages of retrieval.

Diagnose common errors

A local module cannot be imported. Keep `main.py` and its six supporting modules together. Run the illustration from the week folder as instructed.

Judgments refer to documents no longer present. Keep IDs and collection files consistent, rebuild the model, and review judgments after edits. The program reports unknown judgment IDs.

Submit these items

- A complete runnable project folder containing all modules, docs, relevance data, and a README with exact commands.
- A short report explaining the pipeline and the design choices, including preprocessing, weighting, and zero-result behavior.
- Computed rankings and evaluation results for all three supplied judged queries, plus one original judged query.
- A test transcript covering normal search, an unknown query, missing/empty collection handling, and consistent DF/posting counts.

Your evidence should identify the input or query, expected result, actual result, and explanation. State any document edits or preprocessing changes so the result can be reproduced. Include this completed worksheet or a clearly numbered digital response.

Exit questions

Which parts of the model can be reused for several queries?

Why must the model be rebuilt after documents change?

Optional challenge

Use the union of posting lists for nonzero-weight query terms to select ranking candidates. Confirm that positive-score results match the all-document baseline, then compare the number of cosine calculations.

Source for this companion

Based on the supplied Week 10 lab sheet, shipped program, exercise starter, and sample data. The complete main program and `expected_output.txt` in this folder are the references for checking your baseline run.